

StayVeo Tiffin Service — Official Technical Architecture & Dependency Specification

Document Status: Production Official Architecture Specification
Target System: StayVeo Tiffin Domain Engine (Frontend, API Layer, Data Infrastructure, Payment Core, & Operations)
Scope: Tiffin Service Domain ONLY (Strictly excludes PG-specific modules, tables, endpoints, and workflows unless technically shared at the infrastructure level).

1. Executive Summary

The **StayVeo Tiffin Service** is a mission-critical, enterprise-grade daily meal subscription and kitchen operations domain within the StayVeo platform ecosystem. It manages the entire lifecycle of student food subscriptions, kitchen owner onboarding, menu scheduling, automated meal entitlement tracking, payment processing, OTP-based delivery verifications, and kitchen complaint resolutions.

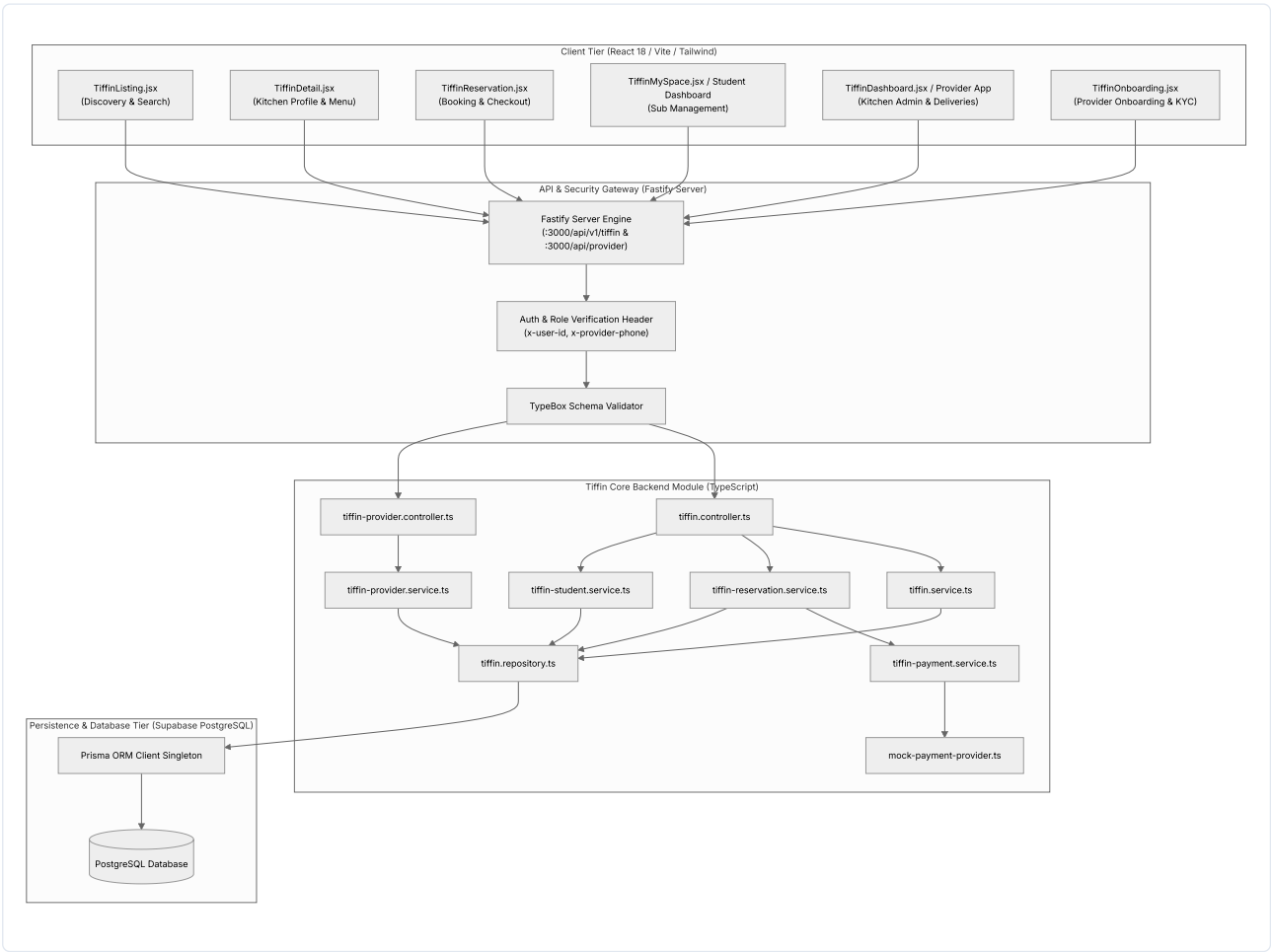


High-Level Domain Capabilities

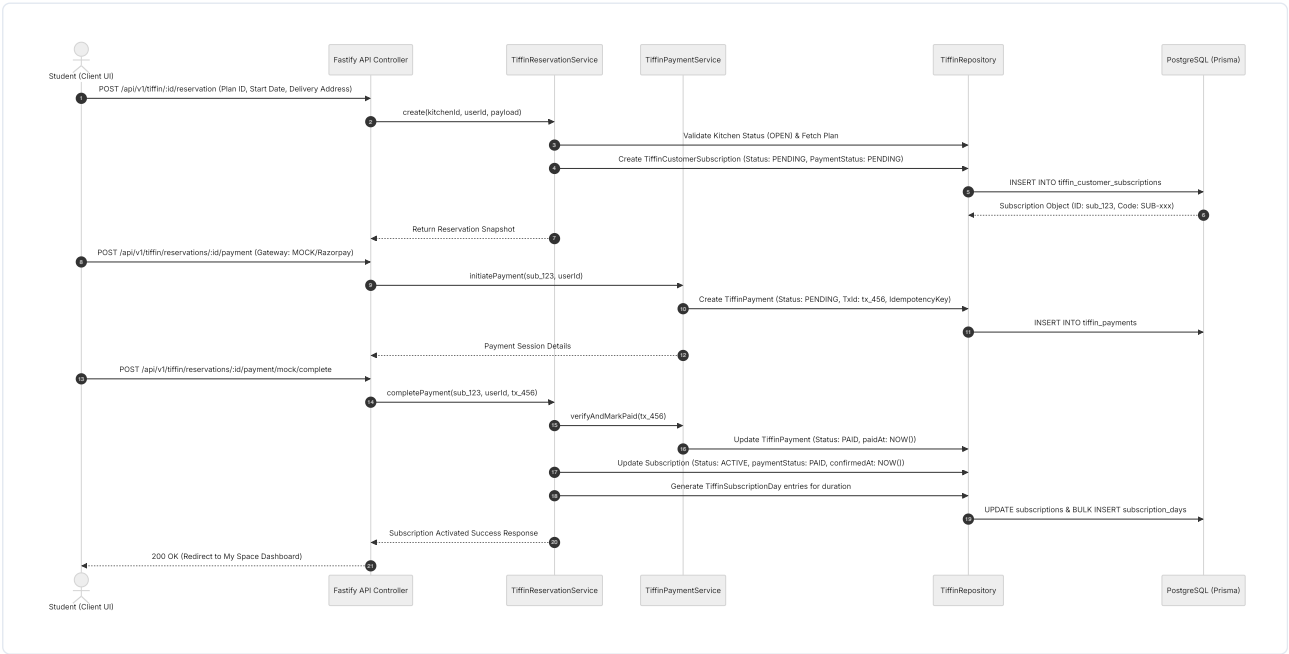
- 1. Student Experience:** Discovery of local kitchens, detailed weekly menu lookup, flexible plan subscriptions (Weekly, Monthly, Custom), real-time meal skipping, and subscription pause/resume controls.
- 2. Kitchen Provider Management:** Multi-step onboarding (address, FSSAI license, operating hours, delivery radius), day-of-week meal timing configuration, weekly menu management with change history, and customer lifecycle management.
- 3. Fulfillment Engine:** Daily subscription day auto-allocation, automated OTP generation for delivery verification, extra-meal tracking, and operational delivery status logs (`SCHEDULED` , `DELIVERED` , `SKIPPED` , `MISSED` , `CANCELLED` , `FAILED`).
- 4. Financial Core:** Multi-stage reservation state tracking (`PENDING` → payment processing → `ACTIVE` subscription), mock payment simulation engine, payment idempotency enforcement, refund logging, and automated invoice code generation.

2. Architecture & System Flow

2.1 System Architecture Diagram



2.2 End-to-End Meal Subscription Sequence

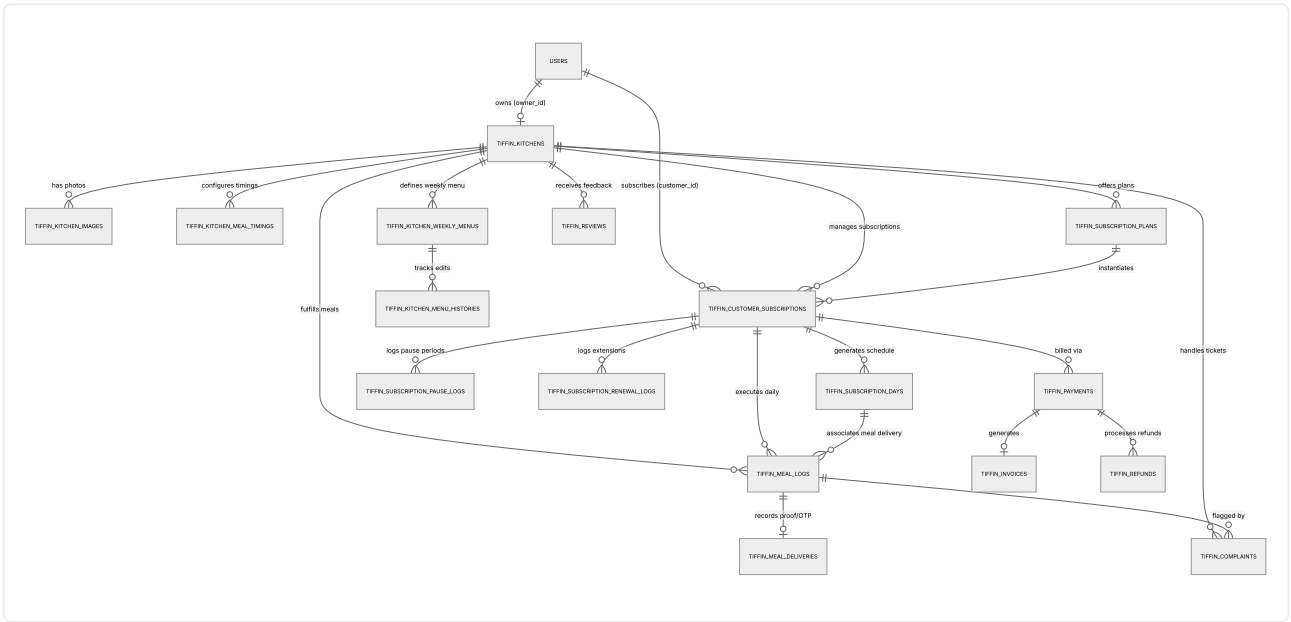


3. Component & Service Mapping Matrix

UI Component / Screen	Frontend Hooks / State	API Endpoint	Method	Backend Handler
TiffinListing.jsx	useFetch , useState(filters)	/api/v1/tiffin	GET	tiffinControll tiffinService.
TiffinDetail.jsx	useParams , useState(kitchen)	/api/v1/tiffin/:id/menu/today	GET	tiffinControll tiffinService.
TiffinReservation.jsx	useNavigate , useState(form)	/api/v1/tiffin/:id/reservation	POST	tiffinControll tiffinReservat
TiffinReservation.jsx	useState(paymentStatus)	/api/v1/tiffin/reservations/:id/payment/mock/complete	POST	tiffinControll tiffinReservat
TiffinMySpace.jsx	useEffect , useState(subscriptions)	/api/v1/tiffin/my-space	GET	tiffinControll tiffinStudents
Student Action Modal	useMutation	/api/v1/tiffin/my-subscriptions/:id/skip-meal	POST	tiffinControll tiffinStudents
Student Action Modal	useMutation	/api/v1/tiffin/my-subscriptions/:id/pause	POST	tiffinControll tiffinStudents
TiffinOnboarding.jsx	useFormContext	/api/provider/onboarding	PUT	tiffinProvider tiffinProvider
TiffinDashboard.jsx	useQuery	/api/provider/dashboard	GET	tiffinProvider tiffinProvider
TiffinDeliveries.jsx	useState(deliveries)	/api/provider/deliveries/mark-all	POST	tiffinProvider → tiffinProvic

4. Complete Data Model & DB Architecture

4.1 Database Entity-Relationship (ER) Diagram



4.2 Comprehensive Data Dictionary

Table 1: `tiffin_kitchens`

Stores registered tiffin kitchen profiles, owner credentials, operating parameters, and cached ratings.

Column Name	Data Type	Nullable	Default	Key / Index	Description
id	UUID	No	gen_random_uuid()	Primary Key	Unique kitchen identifier
owner_id	UUID	No	-	Unique Key, FK (users.id)	Supabase User ID of kitchen operator
kitchen_name	VARCHAR(200)	No	-	Index (with city/status)	Display name of the kitchen
kitchen_logo	TEXT	Yes	-	-	URL to logo asset
cover_image	TEXT	Yes	-	-	URL to cover image asset
description	TEXT	Yes	-	-	Long-form kitchen description
food_options	JSONB	Yes	-	-	Dynamic dietary tag preferences
owner_name	VARCHAR(150)	No	-	-	Full legal name of operator
phone	VARCHAR(15)	No	-	-	Contact mobile number
email	VARCHAR(255)	Yes	-	-	Contact email address
gst_number	VARCHAR(20)	Yes	-	-	Official GSTIN identifier
fssai_license_number	VARCHAR(30)	Yes	-	-	Food safety license registration
address	TEXT	Yes	-	-	Physical kitchen address
latitude	DECIMAL(9,6)	Yes	-	Spatial Index	Geo-latitude coordinate
longitude	DECIMAL(9,6)	Yes	-	Spatial Index	Geo-longitude coordinate
pincode	VARCHAR(10)	Yes	-	-	Postal pin code

Column Name	Data Type	Nullable	Default	Key / Index	Description
city	VARCHAR(100)	Yes	-	Index	Operating city
state	VARCHAR(100)	Yes	-	-	Operating state
delivery_radius_km	DECIMAL(5,2)	No	5.00	-	Max delivery distance limit
food_type	ENUM	No	'veg'	-	veg, nonveg, jain, both
breakfast_available	BOOLEAN	No	false	-	Flag for breakfast service
lunch_available	BOOLEAN	No	true	-	Flag for lunch service
dinner_available	BOOLEAN	No	true	-	Flag for dinner service
meals_per_day	INTEGER	No	2	-	Default offered meal count
security_deposit	DECIMAL(10,2)	No	0.00	-	Refundable deposit required
delivery_charges	DECIMAL(10,2)	No	0.00	-	Per-subscription delivery fee
packing_charges	DECIMAL(10,2)	No	0.00	-	Per-subscription packing fee
extra_meal_price	DECIMAL(10,2)	No	80.00	-	Price for ad-hoc extra meal
is_sunday_closed	BOOLEAN	No	false	-	Flag for Sunday closure
status	ENUM	No	'open'	Index	'open', 'closed', 'temporarily_closed'
verification_status	ENUM	No	'pending'	Index	'pending', 'verified', 'rejected'
average_rating	DECIMAL(3,2)	No	0.00	-	Cached aggregated rating score
total_reviews	INTEGER	No	0	-	Count of verified reviews
created_at	TIMESTAMPTZ	No	now()	-	Audit creation timestamp
updated_at	TIMESTAMPTZ	No	now()	-	Audit last update timestamp
deleted_at	TIMESTAMPTZ	Yes	-	-	Soft deletion timestamp

Table 2: tiffin_subscription_plans

Defines available subscription packages offered by kitchens.

Column Name	Data Type	Nullable	Default	Key / Index	Description
id	UUID	No	gen_random_uuid()	Primary Key	Plan identifier
kitchen_id	UUID	No	-	Foreign Key, Index	FK to tiffin_kitchens.id
plan_name	VARCHAR(150)	No	-	-	e.g. "Monthly Executive Thali"
plan_type	ENUM	No	-	-	'weekly', 'monthly', 'custom'
duration_days	INTEGER	No	-	-	Entitlement validity in days
total_meals	INTEGER	No	-	-	Total meal quota for plan
price	DECIMAL(10,2)	No	-	-	Base price in INR
discount_price	DECIMAL(10,2)	Yes	-	-	Promotional discounted price
is_active	BOOLEAN	No	true	Index	Visibility flag

Table 3: tiffin_customer_subscriptions

Core table managing student subscriptions and active entitlement status.

Column Name	Data Type	Nullable	Default	Key / Index	Description
id	UUID	No	gen_random_uuid()	Primary Key	Unique subscription ID
subscription_code	VARCHAR(30)	No	-	Unique Key	Human-readable identifier (SUB-xxxx)
kitchen_id	UUID	No	-	FK, Composite Index	FK to tiffin_kitchens.id
customer_id	UUID	No	-	FK, Composite Index	FK to users.id
plan_id	UUID	No	-	Foreign Key	FK to tiffin_subscription_plans.id
diet_preference	ENUM	No	'veg'	-	'veg', 'nonveg', 'jain', 'both'
opted_breakfast	BOOLEAN	No	true	-	Opted into breakfast delivery
opted_lunch	BOOLEAN	No	true	-	Opted into lunch delivery
opted_dinner	BOOLEAN	No	true	-	Opted into dinner delivery
delivery_address	TEXT	Yes	-	-	Target delivery location
amount	DECIMAL(10,2)	No	0.00	-	Final billed amount
start_date	DATE	No	-	-	Effective starting date
end_date	DATE	No	-	Index	Expiration date
total_meals_allocated	INTEGER	No	-	-	Initial quota count
meals_remaining	INTEGER	No	-	-	Available active meal count
meals_consumed	INTEGER	No	0	-	Count of delivered meals
meals_skipped	INTEGER	No	0	-	Count of user-skipped meals
status	ENUM	No	'pending'	Index	'pending', 'active', 'paused', 'expired', 'cancelled'
payment_status	ENUM	No	'pending'	-	'pending', 'paid', 'failed', 'refunded'
paused_at	TIMESTAMPZ	Yes	-	-	Timestamp when subscription paused

Table 4: tiffin_meal_logs

Fulfillment tracking table recording individual meal execution status.

Column Name	Data Type	Nullable	Default	Key / Index	Description
id	UUID	No	gen_random_uuid()	Primary Key	Meal tracking ID
subscription_id	UUID	No	-	Foreign Key	FK to tiffin_customer_subscriptions.id
kitchen_id	UUID	No	-	Composite Index	FK to tiffin_kitchens.id
customer_id	UUID	No	-	Composite Index	FK to users.id
meal_date	DATE	No	-	Composite Index	Target meal fulfillment date
meal_category	ENUM	No	-	Composite Index	'breakfast', 'lunch', 'dinner'
status	ENUM	No	'scheduled'	Composite Index	'scheduled', 'delivered', 'skipped', 'missed', 'cancelled', 'failed'
delivery_otp	VARCHAR(6)	Yes	-	-	6-digit OTP for handover proof

Column Name	Data Type	Nullable	Default	Key / Index	Description
delivered_at	TIMESTAMPZ	Yes	-	-	Handover timestamp

Table 5: `tiffin_payments`

Financial transaction ledger for subscriptions.

Column Name	Data Type	Nullable	Default	Key / Index	Description
id	UUID	No	gen_random_uuid()	Primary Key	Payment transaction ID
subscription_id	UUID	No	-	Foreign Key	FK to <code>tiffin_customer_subscriptions.id</code>
kitchen_id	UUID	No	-	Foreign Key, Index	FK to <code>tiffin_kitchens.id</code>
customer_id	UUID	No	-	Foreign Key, Index	FK to <code>users.id</code>
transaction_id	VARCHAR(128)	Yes	-	Unique Key	Gateway transaction reference
idempotency_key	VARCHAR(128)	Yes	-	Unique Key	Unique request key preventing double-charge
payment_gateway	VARCHAR(50)	Yes	-	-	'MOCK', 'RAZORPAY', 'CASHFREE'
payment_method	ENUM	No	'upi'	-	'upi', 'card', 'net_banking', 'wallet', 'cash'
status	ENUM	No	'pending'	Index	'pending', 'processing', 'paid', 'failed', 'cancelled', 'refunded'
total_amount	DECIMAL(10,2)	No	-	-	Total payable amount in INR
paid_at	TIMESTAMPZ	Yes	-	Index	Payment settlement timestamp

5. Detailed API Specifications

5.1 Public / Student Tiffin Endpoints

POST `/api/v1/tiffin`

- **Description:** Creates a new Tiffin Kitchen profile.
- **Headers:** `x-user-id: UUID` (Required)
- **Request Body:**

```
{
  "kitchenName": "Annapurna Pure Veg Tiffin",
  "foodType": "veg",
  "phone": "+919876543210",
  "address": "42 Knowledge Park, Greater Noida",
  "latitude": 28.4744,
  "longitude": 77.5040,
  "deliveryRadiusKm": 5.0,
  "breakfastAvailable": true,
  "lunchAvailable": true,
  "dinnerAvailable": true
}
```

- **Response Headers:** `HTTP/1.1 201 Created`
- **Response Body:**

```
{
  "success": true,
  "message": "Tiffin service created successfully",
  "data": {
    "id": "c7b39a82-1234-4a56-8b90-112233445566",
    "kitchenName": "Annapurna Pure Veg Tiffin",
    "status": "open",
    "verificationStatus": "pending"
  }
}
```

GET /api/v1/tiffin

- **Description:** Search & list active tiffin kitchens with geolocation and food type filters.

- **Query Parameters:**

- `city` (string, optional)
- `foodType` (string: `veg` | `nonveg` | `jain` | `both`, optional)
- `lat` (number, optional)
- `lng` (number, optional)
- `radius` (number, optional, default: `5`)
- `page` (number, optional, default: `1`)
- `limit` (number, optional, default: `10`)

- **Response Body:**

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "c7b39a82-1234-4a56-8b90-112233445566",
        "kitchenName": "Annapurna Pure Veg Tiffin",
        "foodType": "veg",
        "averageRating": 4.8,
        "totalReviews": 34,
        "distanceKm": 1.2,
        "monthlyPrice": 3200.00
      }
    ],
    "meta": { "total": 1, "page": 1, "totalPages": 1 }
  }
}
```

POST /api/v1/tiffin/:id/reservation

- **Description:** Initiates a meal subscription reservation (creates `PENDING` subscription record).
- **Headers:** `x-user-id`: UUID (Required)
- **Path Parameters:** `id`: Kitchen UUID
- **Request Body:**

```
{
  "planId": "p990011-2233-4455-6677-8899aabbccdd",
  "startDate": "2026-09-10",
  "optedBreakfast": true,
  "optedLunch": true,
  "optedDinner": true,
  "deliveryAddress": "Hostel 4, Room 202, Bennett University"
}
```

- **Response Body:**


```
{
  "success": true,
  "message": "Reservation created and awaiting payment",
  "data": {
    "reservationId": "sub_8899aabbccdd",
    "subscriptionCode": "SUB-202609-8899",
    "status": "pending",
    "amount": 3200.00,
    "currency": "INR"
  }
}
```

POST /api/v1/tiffin/reservations/:id/payment/mock/complete

- **Description:** Simulates payment adapter verification & completes subscription activation.
- **Headers:** `x-user-id`: UUID (Required)
- **Path Parameters:** `id`: Reservation/Subscription UUID
- **Request Body:**

```
{
  "transactionId": "tx_mock_99887766",
  "paymentMethod": "upi"
}
```

- **Response Body:**

```
{
  "success": true,
  "message": "Mock payment verified and reservation updated",
  "data": {
    "subscriptionId": "sub_8899aabbccdd",
    "status": "active",
    "paymentStatus": "paid",
    "confirmedAt": "2026-09-06T08:30:00Z"
  }
}
```

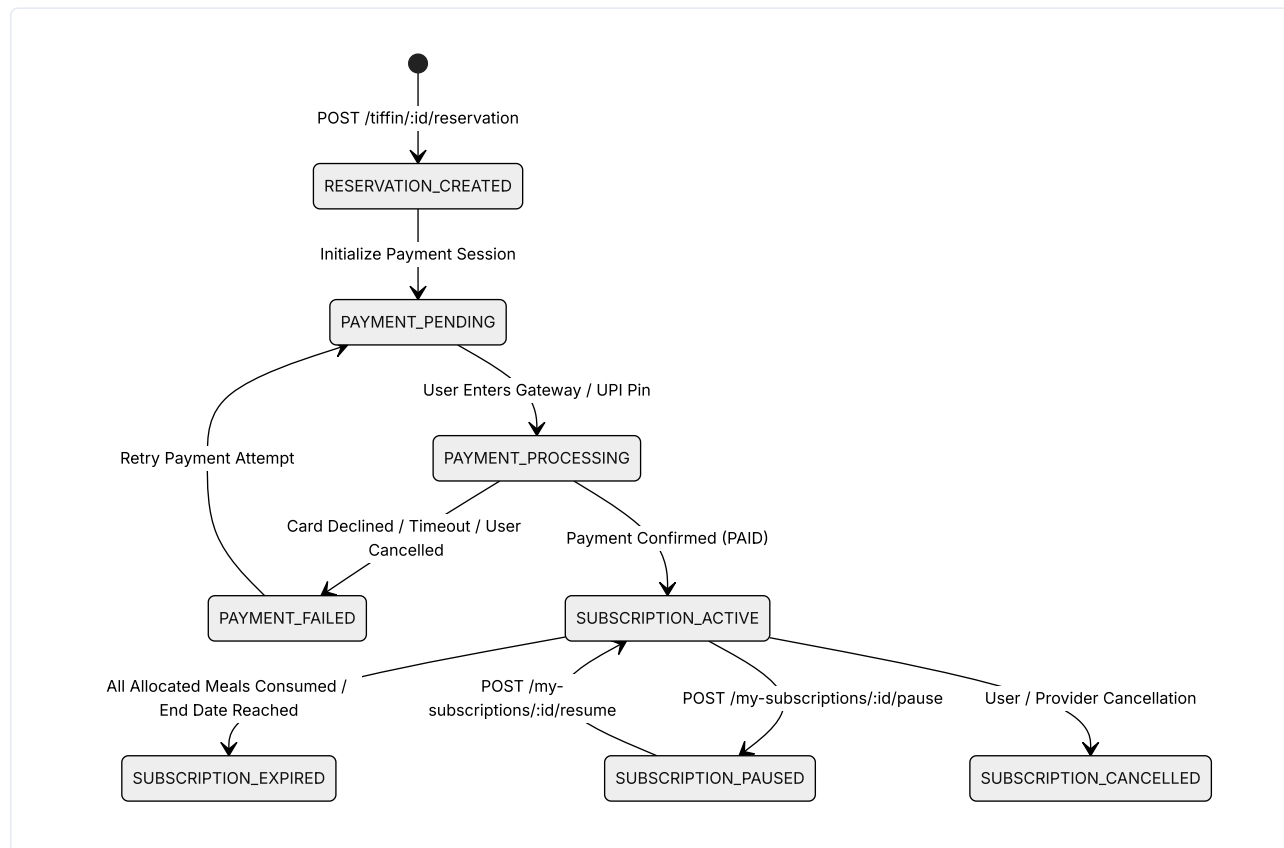
5.2 Provider-Exclusive Management Endpoints

All provider routes automatically extract and verify operator identity via `x-provider-phone` and `x-user-id` HTTP request headers.

- `GET /api/provider/onboarding`: Retrieves provider onboarding draft state.
- `PUT /api/provider/onboarding`: Upserts provider kitchen setup profile.
- `POST /api/provider/onboarding/submit`: Submits kitchen for admin KYC review.
- `GET /api/provider/dashboard`: Computes operational stats (Active Subscriptions, Today's Meals B/L/D, Monthly Revenue).
- `GET /api/provider/deliveries`: Lists scheduled meal deliveries filtered by `date` and `mealCategory`.
- `POST /api/provider/deliveries/mark-all`: Mass-marks scheduled deliveries as `DELIVERED` for selected category.
- `PATCH /api/provider/deliveries/:id`: Updates status or verifies OTP for individual delivery.
- `GET /api/provider/menu`: Fetches configured 7-day weekly menu matrix.
- `PUT /api/provider/menu`: Saves weekly menu items and logs changes to `tiffin_kitchen_menu_histories`.

6. Payment Integration Lifecycle

6.1 Payment & Subscription State Machine



6.2 Payment Adapter Architecture

The StayVeo backend employs a decoupled Payment Gateway Abstraction pattern (`payment-provider.ts` interface) allowing seamless switching between mock testing and live production gateways:

```

export interface PaymentProvider {
  createOrder(params: CreateOrderParams): Promise<OrderResult>;
  verifyPayment(params: VerifyPaymentParams): Promise<VerifyResult>;
  refund(params: RefundParams): Promise<RefundResult>;
}

```

- **Mock Implementation (`mock-payment-provider.ts`)**: Simulates 100% reliable local payment authorization, auto-generating mock transaction IDs (`tx_mock_...`), handling idempotency keys, and updating payment records instantly.
- **Production Transition Strategy**: To switch to Razorpay/Cashfree:
 1. Set `PAYMENT_GATEWAY_PROVIDER=RAZORPAY` in `.env`.
 2. Implement `RazorpayPaymentProvider` adhering to `PaymentProvider` interface.
 3. Register Razorpay webhook endpoint (`/api/v1/tiffin/webhooks/razorpay`) with signature verification using `RAZORPAY_WEBHOOK_SECRET`.

7. Security, Auditability & Compliance

+-----+ SECURITY & AUDIT LAYERS +-----+		
[RBAC / Auth]	Header Authentication (x-user-id, x-provider-phone)	
[Validation]	Fastify Schema / TypeBox Boundary Inspection	
[Data Isolation]	Tenant-scoped Database Queries via Prisma Repo	
[Audit Ledger]	tiffin_activity_logs & tiffin_kitchen_menu_histories	
+-----+		

1. Header-Based Authentication & Ownership Isolation:
- Every API request validates `x-user-id`.
 - Provider handlers extract `x-provider-phone` and resolve provider profile ownership internally.
 - Provider endpoints NEVER accept a `providerId` parameter from client request bodies or path params, preventing Horizontal Privilege Escalation.
2. Input Sanitization & Schema Enforcement:
- All HTTP routes define strict TypeBox JSON Schemas.
 - Malformed payloads, SQL injection strings, or invalid UUIDs are rejected at the Fastify gateway before reaching service handlers.
3. Audit Trails & Change Histories:
- **Menu Integrity:** Edits to weekly menus automatically insert snapshots into `tiffin_kitchen_menu_histories`.
 - **Operational Audit:** Critical lifecycle actions (subscription pause, cancellation, manual delivery overrides) write structured audit rows to `tiffin_activity_logs`.
4. Delivery Verification Security:
- Meals marked delivered via OTP require verification of a cryptographically random 6-digit pin generated during daily log initialization.

8. Scalability, Performance & Infrastructure

1. Database Indexing Strategy:
- Geospatial Indexing on `tiffin_kitchens(latitude, longitude)` for sub-millisecond distance queries.
 - Compound Indexes on `tiffin_meal_logs(kitchen_id, meal_date, meal_category, status)` for instant daily delivery sheet rendering.
 - Unique Index on `tiffin_customer_subscriptions(subscription_code)` for instant lookup.
2. Prisma Connection Pooling:
- Utilizes Supabase PgBouncer pooled connection strings (`DATABASE_URL` with transaction mode) for lightweight HTTP server concurrency.
 - Direct connection string (`DIRECT_URL`) reserved for schema migrations and DDL execution.
3. Automated Daily Meal Scheduler:
- Nightly background process evaluates active subscriptions and generates corresponding `tiffin_meal_logs` rows for the subsequent service day.
 - Automated expiration runner transitions subscriptions with `meals_remaining = 0` or `end_date < CURRENT_DATE` to `EXPIRED`.

9. Maintenance, Operational Playbook & Failure Scenarios

9.1 Runbooks & Common Incident Resolution

Issue 1: Student Claims Meal Was Marked Delivered But Not Received

1. Inspect `tiffin_meal_logs` for `subscription_id` and `meal_date`.
2. Check `tiffin_meal_deliveries` to verify whether `verified_otp` or `proof_image_url` was attached.
3. If delivery was marked without valid OTP, flag kitchen operator via `tiffin_complaints` and credit +1 meal back to subscription `meals_remaining`.

Issue 2: Payment Deducted But Subscription Remains `PENDING`

1. Query `tiffin_payments` by `transaction_id` or `idempotency_key`.
2. If `tiffin_payments.status = 'PAID'` but `tiffin_customer_subscriptions.status = 'PENDING'`, execute manual sync command:

```
npm run task:confirm-reservation -- --reservationId=<UUID>
```

3. The confirmation service idempotent logic will activate the subscription and generate missing `tiffin_subscription_days`.

9.2 Observability & Health Monitoring

- **Health Check Endpoint:** `GET /health` returns status of Fastify server and PostgreSQL connection pool.
- **Log Format:** Structured JSON output using Pino logger with trace context headers (`x-request-id`).

Documentation compiled and verified against StayVeo Tiffin Production Codebase.